

LABORATOR 5

Comunicarea între procese:

Fifo

1. Scopul lucrării

Lucrarea prezintă conductele cu nume (fișiere FIFO) ca un mijloc de comunicație între procese. Acestea vor fi comparate cu conductele fără nume (anonime) și se vor prezenta câteva aplicații tipice pentru acestea.

2. Considerații teoretice

Acest tip de canale nu este susținut de versiunea 7 a sistemului de operare UNIX. În același scop se pot utiliza canalele anonime, însă utilizarea acestora este limitată la procese de tip părinte-fiu sau procese fiu ale aceluiași părinte. Fișierele FIFO combina caracteristicile fișierelor obișnuite cu cele ale fișierelor PIPE. Fișierele FIFO sunt caracterizate de faptul că fiecărui astfel de fișier îi corespunde o intrare în unul din cataloagele sistemului de fișiere lucru care face posibilă referirea unui astfel de fișier prin intermediul numelui asociat acestuia. De asemenea, un fișier FIFO este deschis la fel cu fișierele obișnuite. După deschidere însă, fișierul FIFO capătă caracteristicile unui fișier PIPE. Datele se citesc strict în ordinea în care au fost depuse, iar apelurile de scriere și citire în fișier se garantează a fi atomice. Nu este posibilă poziționarea în fișier utilizând apelul de sistem *lseek*.

Când se deschide un fișier FIFO în citire, se iese din apel doar în momentul în care un alt proces deschide același fișier în scriere. Similar în cazul deschiderii în scriere. Cu alte cuvinte, două procese care comunică printr-un fișier FIFO se așteaptă unul pe celalalt la deschidere, pentru sincronizare, înainte de începerea comunicației propriu-zise. Aceasta așteptare nu mai are loc dacă la deschidere se specifică semaforul `O_NDELAY`, caz în care dacă la celalalt capăt deja exista un proces, atunci se produce sincronizarea, altfel apelul de deschidere este terminat cu eroare.

2.1. Crearea unui fișier FIFO

Pentru a crea un fișier de acest tip este utilizat apelul de sistem **mknod**. Un astfel de canal va exista atâta timp cât nu se realizează un apel al lui **unlink** în scopul ștergerii sale. Apelul de sistem **creat** poate fi utilizat pentru crearea unui fișier obișnuit. Un fișier catalog sau special nu poate fi creat prin acest apel.

Crearea unui fișier FIFO se realizează, în versiunea SVR3, prin apelul funcției de sistem **mknod**. Aceasta funcție de sistem are și rolul de a crea directoare și fișiere speciale. De aceea, în continuare se prezintă interfața sa particularizată pentru crearea fișierelor FIFO:

```
#include <sys/types.h>
#include <sys/stat.h>
int mknod(char *nume_cale, int tip_perm_acces, 0)
Returnează 0 în caz de succes, (-1) în caz de eroare.
```

Argumentul **nume_cale** reprezintă numele de cale al fișierului FIFO, **tip_perm_acces** reprezintă un întreg ce indică tipul fișierului `S_FIFO` (în cazul pipe-urilor cu nume) și drepturile de acces (r,w,x pentru proprietar, grup și ceilalți utilizatori). Acest parametru se dă sub forma:

S_FIFO | drepturi

De exemplu, pentru a crea un fișier FIFO care să aibă drepturile 'read' și 'write' pentru proprietar, grup și alții, se poate folosi funcția:

```
int makepipe( char *path) {  
    return( mknod( path, S_FIFO | 0666, 0));  
}
```

Crearea unui fișier FIFO este similară cu crearea unui fișier ordinar dacă se folosește funcția **mkfifo**, care de fapt apelează tot funcția **mknod**. Interfața funcției **mkfifo**, existentă atât în versiunea SVR4 cât și în versiunea 4.3+BSD este:

```
#include <sys/types.h>
```

```
#include <sys/stat.h>
```

```
int mkfifo(const char *nume_cale, mode_t tip_perm_acces)
```

Returnează 0 în caz de succes, (-1) în caz de eroare.

Semnificația argumentelor este identică cu a celor de mai sus. Ultimul argument lipsește în acest caz.

Comunicarea între procese prin fișiere FIFO se face astfel:

- Un proces creează fișierul FIFO, precizând numele sau simbolic, apelând funcția de sistem **mknod** sau **mkfifo**.
- Procesul care comunica date altora deschide fișierul cu funcția de sistem **open** și scrie datele respective cu funcția **write**.
- Procesul care primește date deschide fișierul FIFO în citire cu funcția de sistem **open** și apoi citește datele din el cu funcția **read**.

3. Aplicații

3.1. Implementarea fișierelor PIPE

O utilizare a fișierelor FIFO ar fi implementarea fișierelor PIPE. În locul apelului *pipe* va fi utilizat apelul *mknod* pentru crearea fișierului FIFO, după care acesta va fi deschis de două ori. Cei doi descriptori de fișier pot fi utilizați exact ca și cei obținuți în urma apelului de sistem *pipe*. O astfel de utilizare este inefficientă. Se pierde mult timp pentru crearea fișierului FIFO și trebuie realizată o gestiune foarte bună a numelor fișierelor FIFO. Din această cauză fișierele FIFO vor fi utilizate în acest scop doar pentru a realiza conexiunea dintre două procese ce nu se află în relația părinte-fiu, sau fii ai aceluiași părinte. O astfel de conexiune nu va putea fi realizată prin fișiere PIPE, datorită limitării cazurilor în care un descriptor de fișier poate fi transmis de la un proces la altul.

3.2. Implementarea mesajelor

Fișierele FIFO pot fi folosite pentru a implementa cozile de mesaje. De fapt fișierele FIFO au fost introduse nu pentru a înlocui pipe-urile ci pentru a înlocui mesajele. Deși în versiunile actuale de Unix mesajele sunt implementate, de multe ori este mai comodă utilizarea fișierelor FIFO în acest scop.

Structura unui mesaj este precizată în fișierul **mesaje.h**.

```
/***** MESAJE.H *****/
```

```
typedef struct {
    long unused; /* diverse informatii asociate */
    int pid; /* identificator proces */
    int number; /* număr mesaj */
} Message;
```

Primul câmp din aceasta structură este lăsat neutilizat având în vedere faptul ca un mesaj este prefixat de tipul său. Asupra acestui câmp se va reveni în lucrarea destinată comunicării prin mesaje.

Pentru transmiterea și recepționarea mesajelor au fost implementate în fișierul **mesaje.c** doua funcții **send** și **receive**. Interfața lor este asemănătoare cu cea a funcțiilor de sistem **write** și **read** după cum se poate observa:

```
int send( int dstkey, char *buf, int nbytes)
```

și

```
int receive( int srckey, char *buf, int nbytes)
```

Primul argument din ambele funcții este folosit la identificarea fișierului, FIFO. Argumentul *buf* refera mesajul, care este de lungime *nbytes*. Se observa ca argumentul *buf* este un pointer la tipul *char* și nu un pointer la structura *Message*, deoarece structura mesajului variază de la o aplicație la alta. Transmiterea unui mesaj într-o coadă plină cauzează blocarea procesului până ce coada deține suficient spațiu pentru a păstra mesajul.

Mesajele sunt trimise/recepționate spre/din cozile de mesaje care sunt identificate printr-o număr întreg numit *cheie*. Mesajele sunt depuse în coadă în ordinea sosirii lor. Coada de mesaje este un fișier FIFO. Numele fișierului FIFO este format dintr-o cheie utilizator care este prefixată de șirul */tmp/fifo*. Acest lucru este realizat de funcția **fifoname**.

Implementarea trebuie să evite două probleme:

- Procesul care transmite mesajul nu trebuie blocat dacă procesul care recepționează mesajul nu a deschis fișierul FIFO pentru citire. Dar nici procesul care transmite mesajul nu trebuie să renunțe la prima încercare nereușită, având în vedere că procesul care recepționează poate să nu fie gata. Așa încât cel care transmite mesajul deschide fișierul FIFO, în scriere cu indicatorul `O_NDELAY` poziționat. Dacă scrierea eșuează deoarece nici un proces cititor nu a deschis fișierul FIFO procesul este pus în așteptare pentru `NAPTIME` secunde. Operația se repetă de `MAXTRIES` ori, după care se renunță.
- Mai multe procese care transmit mesaje pot să deschidă de prea multe ori fișierul FIFO astfel încât limita maximă de descriptori de fișier din sistem să fie depășită. O soluție bună în acest caz este păstrarea a numai `MAXOPEN` fișiere FIFO deschise. La o eventuală cerere suplimentară se închide unul dintre fișierele FIFO deschise și este disponibilizat descriptorul lui. Dacă fluxul de mesaje este de tipul client-server, închiderea fișierului cel mai vechi neutilizat este o abordare convenabilă. Pentru aceasta, se va păstra o tabelă cu descriptorii utilizați (**fifos**). Fiecare intrare din tabelă deține în câmpul **time** timpul relativ la care a fost folosit descriptorul. La fiecare apel **send** sau **receive** se va incrementa valoarea variabilei locale **clock** și valoarea ei va atribuită câmpului **time**. La închiderea unui descriptor este suficientă inspectarea tabelii și găsirea celui cu valoarea din câmpul **time** cea mai mare. Funcția care gestionează descriptorii este **openfifo**.

Codul acestor funcții ilustrat în cele ce urmează prin doua programe: *speak.c*, *tick.c*

Programul *peak.c* trimite date prin FIFO catre cel de al doilea program *tick.c* care extrage datele si le afiseaza. Programul *peak.c* creaza FIFO numit *comunicatie*, apoi deschide acest FIFO, operatia `open()` blocheaza procesul pana la aparitia acestora unui proces care extrage datele din FIFO.

//peak.c

```
#include <stdio.h>
#include <stdlib.h>
#include <errno.h>
#include <string.h>
#include <fcntl.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <unistd.h>
#define FIFO_NAME "comunicatie"
int main(void)
{
    char s[300];
    int num, fd;
    mknod(FIFO_NAME, S_IFIFO | 0666, 0);

    printf("waiting for readers...\n");

    fd = open(FIFO_NAME, O_WRONLY);
    printf("got a reader--type some stuff\n");
    while (gets(s), !feof(stdin)) {
        if ((num = write(fd, s, strlen(s))) == -1)
            perror("write");
        else
            printf("speak: wrote %d bytes\n", num);
    }
    return 0;
}
```

Programul al doilea *tick.c*, ca si primul program, blocheaza procesul in cazul in care nu exista date scrise in FIFO. Imediat ce apare un proces care scrie date in FIFO, programul *tick.c* intra in executie. Este important de precizat ca pot exista mai multe procese care scriu in acelasi FIFO.

//tick.c

```
#include <stdio.h>
#include <stdlib.h>
#include <errno.h>
#include <string.h>
#include <fcntl.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <unistd.h>
#define FIFO_NAME "comunicatie"
int main(void)
{
    char s[300];
    int num, fd;
    mknod(FIFO_NAME, S_IFIFO | 0666, 0);
    printf("waiting for writers...\n");
    fd = open(FIFO_NAME, O_RDONLY);
    printf("got a writer\n");
    do {
```

```

if ((num = read(fd, s, 300)) == -1)
perror("read");
else {
s[num] = '\0';
printf("tick: read %d bytes: \"%s\"\n", num, s);
}
} while (num > 0);
return 0;

}

```

Observatie: In cazul in care executia programului tick.c este intrerupta in timp de speak.c functioneaza, poate rezulta intr-un mesaj " Broken Pipe". Acesta apare in cazul in care toate procesele care citesc sunt terminate, iar procesul care scrie este inca in executie. Priocetul care scrie va receptiona semnalul SIGPIPE data urmatoare cand incearca sa scrie date in FIFO. In cazul in care se trateaza acest semnal, procesul poate afisa un mesaj de eroare extins.

Funcția **rmqueue** are rolul de a șterge fișierul FIFO. Aceasta funcție este apelată de programul următor.

```

/*
A se compila cu: gcc o d delete.c mesaje.c err.o
***** DELETE.C *****/

#include "mesaje.h"

main( int argc, char *argv[])
{
    if ( argc < 2)
        err_sys("Utilizare: d <cheie_FIFO>");
    rmqueue( atoi(argv[1]));
}

```

Pentru a testa funcțiile **send** și **receive**, se consideră doua programe: unul care trimite mesaje - **send.c** - și altul care recepționează mesajele - **receive.c**. Există un singur fișier FIFO în care procesele client scriu mesajul și din care procesul server citește mesajele și le afișează. Codul lor este:

```

/*
A se compila cu: gcc o r receive.c err.o
A se lansa în executie cu: r &
***** RECEIVE.C *****/

#include "mesaje.c"
#include "mesaje.h"

main( void)
{
    Message m;
    setbuf( stdout, NULL);
    while ( receive( 5, ( char *) &m, sizeof( m)))
        printf("Receptionat %d de la %d\n", m.number, m.pid);
    err_sys( "Eroare receive");
}

```

```

/* A se compila cu: gcc o s send.c err.o
A se lansa în executie cu: s [& s ...]
***** SEND.C *****/
#include "mesaje.c"
#include "mesaje.h"

main( void)
{
    Message m;
    m.pid = getpid();
    for( m.number = 1; m.number <= MAXMSG; m.number++) {
        sleep(1);
        if( !send( 5, (char *) &m, sizeof( m)))
            err_sys("Eroare send");
    }
    exit( 0 );
}

```

Cheia cozii de mesaje este fixată în acest caz la 5. Pentru a șterge fișierul FIFO cu cheia 5 se execută comanda **d 5**, iar pentru a distruge procesul care receptionează mesajele se folosește comanda Unix **kill <pid>**.

3.3. Comunicare client - server folosind canale cu nume

O utilizare frecvent întâlnită a fișierelor FIFO este pentru asigurarea căilor de comunicare între un server și diferiți clienți. Dacă există un server contactat de mai mulți clienți, fiecare client poate să-și transmită cererea printr-un fișier FIFO, având nume bine precizat și creat de către server. Problema depunerii cererilor se poate rezolva simplu, printr-un singur fișier, în schimb pentru a furniza răspunsurile, serverul nu poate utiliza un singur fișier FIFO, deoarece fiecare client ar trebui să știe când să-și preia datele din acest fișier. Există mai multe soluții pentru a trimite răspunsurile clienților.

O prima variantă este ca fiecare cerere să fie însoțită de ID procesului care a depus-o. Serverul, la furnizarea unui răspuns, va depune răspunsul într-un fișier FIFO cu numele */tmp/server.nnnn*, unde *nnnn* este ID clientului care așteaptă răspunsul. În aceasta varianta serverul nu-și poate da seama dacă clientul dispăre. Din această cauză pot să rămână fișiere FIFO neșterse în sistemul de fișiere.

Serverul trebuie să capteze semnalul SIGPIPE, deoarece un client poate să trimită o cerere și să se termine înaintea preluării răspunsului, lăsând fișierul FIFO ce-i corespunde cu un proces care scrie în el și fără proces cititor.

O altă variantă ar fi ca fiecare cerere să fie însoțită de ID procesului care a depus cererea, iar ca răspuns să se utilizeze tot un fișier FIFO, având numele bine fixat. Serverul depune întotdeauna răspunsul în acest fișier FIFO, după care anunță printr-un semnal clientul că acesta trebuie să-și preia datele. Deși semnalele nu reprezintă un mijloc de comunicare puternic între procese, combinarea acestora cu fișiere pipe sau FIFO, permite rezolvarea comodă a multor probleme de comunicare.

În continuare este ilustrat codul a două programe, client și server, care comunică între ele cu ajutorul a două fișiere FIFO și a semnalelor. Procesul server deservește cererile de citire dintr-un fișier (**baza_d.txt**), cereri formulate de mai multe procese client. Orice cerere este depusă în fișierul specificat prin constanta simbolică CNAME (Control), iar răspunsurile sunt furnizate

prin fișierul FIFO specificat de constanta simbolică DNAME (Date). Orice depunere de date într-un fișier FIFO este însoțită de anunțarea procesului destinat prin trimiterea către acesta a semnalului SIGUSR1.

Pachetul de cereri este definit în fișierul **packet.h**.

```
/* PACKET.H */
#define READ 0 /* cerere de citire din fișier */
#define CONNECT 1 /* cerere de conectare la server */
#define SENDPID 2 /* trimitere identificator proces */

#define DBUF 128 /* dimensiunea buffer de date */

#define DNAME "/tmp/datapipe" /* FIFO date=raspunsuri */
#define CNAME "/tmp/ctrlpipe" /* FIFO control=cereri */

struct packet { /* pachet de cerere */
    int pk_pid;
    int pk_blk;
    int pk_code;
};
```

Câmpul **pk_blk** precizează numărul blocului de dimensiune DBUF implicat în operația al cărei cod este indicat de câmpul **pk_code**. Cererea este formulată de procesul cu pid-ul precizat de **pk_pid**.

```
/* A se compila cu: gcc o s server.c err.o
A se lansa în executie cu: s&
***** SERVER.C *****/
#include <signal.h>
#include <fcntl.h>
#include <sys/stat.h>
#include "packet.h"
#include "hdr.h"

int datapipe, ctrlpipe, datafile, OK;
char *dataname = "baza_d.txt";

void handler( int sem)
{
    if ( sem==SIGTERM) {
        unlink( DNAME); unlink( CNAME);
        printf("Serverul a fost oprit\n");
        kill( getpid(), SIGTERM);
    }
    signal( SIGUSR1, handler);
    OK++;
}

/* funcția de tratare a cererilor adresate serverului */
int process( struct packet *pkp, struct packet *spkp)
{
    char pbuf[DBUF];
    int no;
    if ( (datapipe = open( DNAME, O_WRONLY | O_NDELAY)) < 0)
        err_sys("Eroare la deschidere datapipe");
    switch( pkp->pk_code ) {
```

```

        case CONNECT: /* conectare client la server */
            spkp>pk_pid = getpid();
            if ( write( datapipe, spkp, sizeof( struct packet))
                != sizeof( struct packet))
                err_sys("Eroare scriere pachet");
            printf("Serverul %d și clientul %d sau conectat\n",
                pkp>pk_pid, spkp>pk_pid);
            break;
        case READ: /* citire bloc de date */
            lseek( datafile, pkp>pk_blk*DBUF,0 );
            no = read( datafile, pbuf, DBUF );
            write( datapipe, pbuf, no);
            break;
        default: /* cerere necunoscuta */
            err_quit("Cod de cerere necunoscut\n" );
    }
    OK = 0;
    kill( pkp>pk_pid, SIGUSR1 );
    while( !OK ); /* asteptare confirmare primire */

    close( datapipe);
}

main()
{
    struct packet pk, sendpk;
    unlink( DNAME ); unlink( CNAME );

    if( mkfifo( DNAME, 0666) < 0 || mkfifo( CNAME, 0666) < 0 )
        err_sys("Nu se pot crea fișierele FIFO !\n");

    ctrlpipes = open( CNAME, O_RDONLY);
    datafile = open( dataname, 0 );

    if( ctrlpipes < 0 || datafile < 0 )
        err_sys("Nu pot deschide ctrl FIFO sau fișierul de date!");

    sendpk.pk_code = SENDPID; /* serverul se face cunoscut */
    signal( SIGUSR1, handler);
    signal( SIGTERM, handler);
    for ever
        while ( read( ctrlpipes, &pk, sizeof( pk ) ) )
            process( &pk, &sendpk );
}

```

Codul pentru procesul client este:

```

/* A se compila cu: gcc o c client.c err.o
A lansa în executie cu: c <nr> [[; c <nr>...] | [& c <nr>]]
***** CLIENT.C *****/
#include <signal.h>
#include <fcntl.h>
#include <sys/stat.h>
#include "packet.h"

int datapipes, ctrlpipes, OK;

```



```

void handler( int sem)
{
    signal( SIGUSR1, handler);
    OK++;
}

/* conectare cu serverul */
int connect()
{
    struct packet pk;
    ctrlpipes = open( CNAME, O_WRONLY | O_NDELAY);
    datapipes = open( DNAME, O_RDONLY | O_NDELAY);
    if ( datapipes < 0)
        err_sys("Nu pot deschide datapipes !");
    if ( ctrlpipes < 0 )
        err_sys("Nu pot deschide ctrlpipes!");

    pk.pk_pid = getpid();
    pk.pk_code = CONNECT;
    OK = 0; /* sa depus cererea */
    write( ctrlpipes, &pk, sizeof( pk));
    while( !OK); /* asteptare raspuns de la server */

    read( datapipes, &pk, sizeof( pk)); /* citire raspuns */
    kill( pk.pk_pid, SIGUSR1); /* confirmare raspuns */
    return( pk.pk_pid);
}

/* cerere bloc de date de la server */
request( char *ptr, int blk, int spid)
{
    struct packet pk;
    int no;
    pk.pk_pid = getpid();
    pk.pk_blk = blk;
    pk.pk_code = READ;
    OK = 0;
    write( ctrlpipes, &pk, sizeof( pk));
    while( !OK );
    no = read( datapipes, ptr, DBUF );
    kill( spid, SIGUSR1 );
    return no;
}

int main( int argc, char *argv[])
{
    int spid;
    int blk;
    char buffer[DBUF];
    int no;
    blk = atoi( argv[1]);
    signal( SIGUSR1, handler); /* armare */
    spid = connect();
    no = request( buffer, blk, spid );
    write( 1, buffer, DBUF);
    printf("\n");
}

```

```
}
```

După lansarea serverului în background fiecare client poate face o cerere de citire a unui bloc de dimensiune DBUF, bloc care este afișat. Serverul este distrus prin transmiterea semnalului SIGTERM către acesta, moment în care se șterg și fișierele FIFO.

4. Probleme propuse

4.1. Să se scrie două programe ce se lansează independent și între care se stabilește un canal FIFO care realizează comunicația bidirecțională între programe.

4.2. Să se scrie un program care echivalează linia de comandă shell: **comanda1 | comanda2** , utilizând un fișier FIFO.

4.3. Să se scrie funcțiile de sincronizare pentru procese părinte fiu folosind fișiere FIFO.

4.4. Să se completeze aplicația [3.3](#), astfel încât să permită și alte operații (scriere, adăugare, ...) în fișier.

4.5. Să se modifice aplicația [3.3](#), astfel încât pentru răspuns să se utilizeze câte un fișier FIFO pentru fiecare client.

4.6. Să se scrie un program care furnizează operații pe două numere. Acesta funcționează pe post de server recepționează cererile de la procesele client și transmite rezultatul înapoi. Structura unui mesaj este:

```
typedef struct {  
    long unused;  
    long clientkey;  
    int x; int y; int rez;  
} Message;
```

4.7. Să se scrie programele client și server pentru a realiza accesul la o baza de date. Aceasta va fi compusă din mai multe fișiere, iar fiecare client poate realiza operațiile citire și scriere relație într-un anumit fișier baza de date.